

Sports Exercise Battle - Protokoll

1. Projektübersicht

SEB ist eine Java-Konsolenanwendung, die einen HTTP/REST-Server für Push-Up-Turniere implementiert. Benutzer können sich registrieren, einloggen, Push-Up-Einträge erfassen und in 2-Minuten-Turnierfenstern gegeneinander antreten. Der Server verteilt ELO-Punkte nach jedem Turnierende und speichert alle Daten in einer PostgreSQL-Datenbank.

Spielmechanik:

- Jeder POST /history Eintrag startet ein neues 2-Minuten-Turnier oder nimmt an einem laufenden teil
 - Gewinner (meiste Push-Ups gesamt): +2 ELO. Unentschieden: alle Gleichplatzierten +1 ELO. Verlierer: -1 ELO
 - Start-ELO: 100 für jeden neuen Benutzer
-

2. Architektur & Designentscheidungen

Das Projekt folgt einer Schichtenarchitektur mit strikten einseitigen Abhängigkeiten:

Schicht	Klassen	Verantwortung
Presentation	Router, SebHttpHandler, *RequestHandler	HTTP parsen, JSON serialisieren
Service	UserService, HistoryService, ...	Dünne Delegations-Fassade
Domain	UserManager, TournamentManager, StatsManager	Business-Logik, ELO-Regeln
Persistence	UserRepository, HistoryRepository, ...	JDBC SQL, DB-Zugriff

Wichtige Designentscheidungen:

Token-Format: Tokens folgen dem Muster `<username>-sebToken`. Der Authorization-Header verwendet Bearer (REST-Standard) statt Basic.

Lazy Tournament Closing: Es wird kein Hintergrund-Timer verwendet. Abgelaufene Turniere werden beim nächsten eingehenden Request geschlossen.

Passwort-Hashing: jBCrypt wird verwendet.

Direktes ELO-Update: ELO wird via `UPDATE users SET elo = elo + ?` direkt in der Datenbank angepasst statt das Objekt zu laden und zurückzuspeichern. Das vermeidet Race Conditions wenn mehrere Turniere gleichzeitig enden.

Multi-Exercise-Unterstützung (Unique Feature): History-Einträge enthalten ein `name`-Feld (z.B. PushUps, Squats) das es Benutzern erlaubt beliebige Übungstypen zu erfassen. Die Turnier- und ELO-Mechanik gilt universell.

Datenbankschema: Drei Tabellen werden beim Start via `CREATE TABLE IF NOT EXISTS` erstellt: `users`, `tournaments`, `history`. Wichtig: history referenziert beide anderen Tabellen über Foreign Keys mit `ON DELETE CASCADE` auf `user_id`. Die Erstellungsreihenfolge ist entscheidend — `users` und `tournaments` müssen vor `history` existieren.

Repository-Schicht: Jedes Repository verwendet einfaches JDBC mit PreparedStatements zur Vermeidung von SQL-Injection.

JSON-Mapping-Problem (Fehler & Lösung): Anfänglicher Fehler: alle Registrierungen gaben "Username must not be blank" zurück. Ursache: curl-Skript sendete `{"Username": "alice"}` mit großem U, aber Jackson mappt standardmäßig auf Kleinbuchstaben-Feldnamen. Lösung: alle JSON-Schlüssel im curl-Skript auf Kleinbuchstaben geändert (username, password, count, duration, name).

Authorization-Header: Das originale curl-Skript verwendete `Authorization: Basic <token>`. Geändert auf `Bearer` was der korrekte HTTP-Standard für Token-basierte Authentifizierung ist.

Lombok + Maven: In IntelliJ funktionierte Lombok über das IDE-Plugin. Aber `mvn clean package` scheiterte mit "Symbol not found: getId()" weil Maven Lombok nicht als Annotation-Processor kannte. Lösung: `<annotationProcessorPaths>` mit Lombok zum `maven-compiler-plugin` in `pom.xml` hinzugefügt.

3. REST-Endpoints

Methode	Pfad	Auth	Beschreibung
POST	/users	Nein	Neuen Benutzer registrieren
POST	/sessions	Nein	Login, gibt Token zurück
DELETE	/sessions	Ja	Logout, widerruft Token
GET	/users/{username}	Ja	Eigenes Profil anzeigen
PUT	/users/{username}	Ja	Eigenes Profil bearbeiten
GET	/history	Ja	Eigene History-Einträge anzeigen

POST	/history	Ja	Eintrag hinzufügen, Turnier starten/beitreten
GET	/stats	Ja	ELO + gesamte Push-Up-Anzahl
GET	/score	Ja	Globales Scoreboard
GET	/tournament	Ja	Aktuelles/letztes Turnier

4. Unit Tests

28 Unit-Tests decken ausschließlich die Domain-Schicht ab, da dort die gesamte Business-Logik liegt. Mockito wird verwendet, um Repository-Abhängigkeiten zu mocken, sodass keine Datenbankverbindung benötigt wird.

Testklasse	Tests	Warum kritisch
TokenManagerTest	7	Token-Sicherheit ist die Grundlage aller authentifizierten Endpoints. Null-Sicherheit für ConcurrentHashMap ist kritisch.
UserManagerTest	10	Register/Login/Logout sind Einstiegspunkte für jeden Benutzer. Passwort-Hashing, Duplikat-Erkennung und Owner-Only-Profilbearbeitung schützen die Datenintegrität.
TournamentManagerEloTest	6	ELO-Verteilung ist die zentrale Spielmechanik. Sonderfälle (Unentschieden, einzelner Teilnehmer, mehrere Einträge pro User) müssen alle korrekt funktionieren.
StatsManagerTest	5	Stats und Scoreboard-Sortierung beeinflussen das sichtbare Ranking. Falsche Sortierung oder Null-Behandlung würden falsche Ergebnisse liefern.

5. Integrationstests (curl-Skript)

Das curl-Skript `seb_curl.bat` deckt 21 Testschritte mit Kommentaren zu erwarteten Ausgaben ab.

Wichtige Anpassungen am originalen Skript:

- `Authorization: Basic` auf `Authorization: Bearer` geändert (korrekter HTTP-Standard)

- Alle JSON-Schlüssel auf Kleinbuchstaben geändert (username, password, count, duration, name)
- Kommentare mit erwarteten Ausgaben vor jedem curl-Aufruf hinzugefügt
- Schritt 20 (Logout) und Schritt 21 (unbekannte Route / 404) hinzugefügt
- Negative Testfälle ergänzt: leerer Username, ungültiger Token, falsches Passwort

Hinweis: Die Datenbank muss zwischen Testläufen geleert werden (`DELETE FROM history; DELETE FROM tournaments; DELETE FROM users;`) um die erwarteten ELO-Werte zu erhalten.

Gesamter Zeitaufwand: ca. 22h